

---

# Lab 03: Configuring AWS CLI and GitHub Inside Your Docker Container

---

**Course:** Cloud Computing **Estimated Time:** 60–75 minutes **Difficulty:** Beginner **Prerequisites:** Lab 01 (AWS Account, IAM User, Access Key) and Lab 02 (Docker Linux Container Setup) must be completed first.

---

## 1. Learning Objectives

---

By the end of this lab, you will be able to:

1. Install the AWS CLI **inside** your Ubuntu Docker container.
  2. Configure the AWS CLI using the IAM lab user's access key from Lab 01.
  3. Verify the CLI can authenticate and query your AWS account.
  4. Install and configure **Git** inside the container using your **college student ID/email**.
  5. Authenticate to **GitHub** from inside the container using a Personal Access Token (PAT).
  6. Clone a repository, commit a script, and push it back to GitHub — all from inside the container.
  7. Make your setup **persistent**, so you don't lose it every time the container restarts.
- 

## 2. Why This Matters

---

Right now, your Ubuntu container is a clean, empty Linux environment. In real cloud engineering work, this is exactly how you'd set up a **reproducible development environment** — install only the tools you need, connect it to your cloud account, and connect it to version control so your work is backed up and gradable. By the end of this lab, your container becomes a mini “workstation” where you can write scripts, run them against your AWS account, and push them to GitHub for submission.

⚠ **Security Reminder:** You will be placing AWS credentials and a GitHub token inside this container. Never commit these to a public repository, never share them with classmates, and never paste them into chat tools. Treat them like passwords.

---

## 3. Prerequisites Checklist

---

Before starting, confirm you have:

- ☐ Completed **Lab 01:** You have an IAM user (not root) with an **Access Key ID** and **Secret Access Key** saved from the downloaded `.csv` file.
  - ☐ Completed **Lab 02:** Docker Desktop installed, and you know how to run `docker start -ai my-ubuntu` or create a new container.
  - ☐ A **GitHub account** created using your **college student ID or college email** (e.g., `yourid@college.edu`). If you don't have one yet, create it now at <https://github.com/signup>.
  - ☐ Your AWS access key `.csv` file and GitHub login handy (but not pasted anywhere public).
- 

## 4. Part A — Start Your Container

---

If your container from Lab 02 was removed, create a fresh one. If it still exists, restart it.

**Option 1 — Reuse existing container:**

```
docker start -ai my-ubuntu
```

**Option 2 — Create a new container (if `my-ubuntu` no longer exists):**

```
docker run -it --name my-ubuntu ubuntu:22.04 /bin/bash
```

✓ **Checkpoint:** Your terminal prompt should show something like `root@<container-id>:/#`.

---

## 5. Part B — Install Base Tools

---

Inside the container, update package lists and install the tools you'll need:

```
apt update && apt upgrade -y
apt install -y curl unzip git nano
```

- `curl` - used to download the AWS CLI installer
- `unzip` - needed to extract it
- `git` - for GitHub version control
- `nano` - a simple text editor (or use `vim` if you prefer)

## 6. Part C — Install AWS CLI v2

1. Download the AWS CLI installer:

```
curl "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" -o "awscliv2.zip"
```

**Note:** If your Mac uses Apple Silicon and Docker is emulating ARM64, use this URL instead: <https://awscli.amazonaws.com/awscli-exe-linux-aarch64.zip>

2. Unzip and install:

```
unzip awscliv2.zip
./aws/install
```

3. Verify installation:

```
aws --version
```

Expected output similar to:

```
aws-cli/2.x.x Python/3.x.x Linux/x.x.x exe/x86_64.ubuntu.22
```

✓ **Checkpoint:** `aws --version` runs without error.

## 7. Part D — Configure AWS CLI with Your IAM Lab User

You will now connect the CLI to your AWS account using the **IAM user's** access key from Lab 01 — **never the root user's credentials**.

1. Run:

```
aws configure
```

2. Enter the following when prompted:

```
AWS Access Key ID [None]: <paste your Access Key ID>
AWS Secret Access Key [None]: <paste your Secret Access Key>
Default region name [None]: us-east-2
Default output format [None]: json
```

Use region `us-east-2`.

3. **Verify authentication** by running a harmless read-only command:

```
aws sts get-caller-identity
```

Expected output:

```
{
  "UserId": "AIDAXXXXXXXXXXXXXX",
  "Account": "123456789012",
  "Arn": "arn:aws:iam::123456789012:user/lab-user"
}
```

Confirm the `Arn` ends with your **IAM lab username**, not `root`.

✓ **Checkpoint:** `aws sts get-caller-identity` returns your IAM user's ARN.

🔔 If you see an `AccessDenied` or `InvalidClientTokenId` error, double-check you copied the keys correctly (no extra spaces) and that the key hasn't been deactivated. See Section 11.

## 8. Part E — Configure Git with Your College Identity

Set your Git identity so every commit you make is correctly attributed to you.

```
git config --global user.name "Your Full Name"
git config --global user.email "yourcollegeid@college.edu"
```

Verify:

```
git config --global --list
```

You should see both `user.name` and `user.email` listed correctly.

## 9. Part F — Authenticate to GitHub with a Personal Access Token (PAT)

GitHub no longer accepts account passwords for Git operations over HTTPS, so you'll create a **Personal Access Token (PAT)** instead.

### F1. Generate a PAT on GitHub (do this in your browser, not the container)

1. Sign in to GitHub with your student account.
2. Click your profile photo → **Settings**.
3. Scroll to **Developer settings** (bottom of left sidebar).
4. Click **Personal access tokens** → **Tokens (classic)** → **Generate new token (classic)**.
5. Give it a note, e.g., `cloud-computing-container`.
6. Set **Expiration** to a reasonable value (e.g., 90 days — check your institution's policy on token lifetimes).
7. Under **Select scopes**, check `repo` (this allows push/pull to your repositories).
8. Click **Generate token**.
9. **Copy the token immediately** — GitHub shows it only once. Save it somewhere secure (a password manager), not in a plain text file on your Desktop.

### F2. Use the PAT Inside the Container

You do **not** put the token into a config file directly. Instead, Git will prompt for it the first time you push or pull:

```
git clone https://github.com/<your-username>/<your-repo>.git
```

- When prompted for a **username**, enter your GitHub username.
- When prompted for a **password**, paste your **PAT** (not your GitHub account password).

To avoid re-entering the token every time during this session, enable Git's credential cache:

```
git config --global credential.helper 'cache --timeout=3600'
```

This caches your credentials in memory for 1 hour (3600 seconds) and clears them automatically — safer than storing them permanently in plain text.

✓ **Checkpoint:** You successfully cloned a repository without a permanent password stored on disk.

## 10. Part G — Write, Run, and Push a Script

Let's tie it all together: write a script that uses the AWS CLI, then push it to GitHub.

1. Move into your cloned repo folder:

```
cd <your-repo>
```

2. Create a simple script:

```
nano list_buckets.sh
```

Paste in:

```
#!/bin/bash
echo "Listing S3 buckets in your AWS account:"
aws s3 ls
```

Save and exit ( `Ctrl+O` , `Enter` , `Ctrl+X` in nano).

3. Make it executable and run it:

```
chmod +x list_buckets.sh
./list_buckets.sh
```

(It's fine if the list is empty — that just means you have no S3 buckets yet.)

4. Stage, commit, and push:

```
git add list_buckets.sh
git commit -m "Add script to list S3 buckets"
git push
```

Enter your GitHub username and PAT again if prompted (unless still cached).

✓ **Checkpoint:** Refresh your repository on GitHub.com in a browser — `list_buckets.sh` should now appear there.

## 11. Troubleshooting

| Issue                                                                | Likely Cause                                                        | Fix                                                                                             |
|----------------------------------------------------------------------|---------------------------------------------------------------------|-------------------------------------------------------------------------------------------------|
| <code>aws: command not found</code>                                  | Install didn't complete, or new shell session                       | Re-run <code>./aws/install</code> , confirm with <code>which aws</code>                         |
| <code>Unable to locate credentials</code>                            | <code>aws configure</code> not run, or run in a different container | Re-run <code>aws configure</code> in the current container                                      |
| <code>AccessDenied</code> on <code>get-caller-identity</code>        | Wrong or deactivated access key                                     | Verify key in AWS Console → IAM → Security credentials                                          |
| <code>remote: Support for password authentication was removed</code> | Used GitHub password instead of PAT                                 | Use your PAT as the password when prompted                                                      |
| <code>fatal: could not read Username</code>                          | Cloning via SSH URL instead of HTTPS, or non-interactive shell      | Ensure repo URL starts with <code>https://github.com/...</code>                                 |
| Credentials keep being asked every command                           | Credential cache expired or not set                                 | Re-run the <code>credential.helper cache</code> command from Part F2                            |
| All work disappears after closing terminal                           | Container was removed ( <code>docker rm</code> ), not just stopped  | Use <code>docker start -ai my-ubuntu</code> instead of creating a new container; see Section 12 |

## 12. Making Your Setup Persistent (Important!)

Docker containers keep their filesystem changes as long as you **stop** them ( `docker stop` ) rather than **remove** them ( `docker rm` ). To resume exactly where you left off:

```
docker start -ai my-ubuntu
```

If you accidentally remove the container, you will need to repeat Parts B-F.

💡 **Tip:** Never store your AWS secret key or GitHub PAT in a file you commit to Git. If you want to reuse them across sessions without retyping, add file detail to `.gitignore`

---

## 13. Submission Checklist

Submit a short lab report including:

- ☐ Screenshot of `aws --version` output.
- ☐ Screenshot of `aws sts get-caller-identity` showing your IAM username in the ARN (not root).
- ☐ Screenshot of `git config --global --list` showing your name and college email.
- ☐ Screenshot of your GitHub repository showing `list_buckets.sh` successfully pushed.
- ☐ One paragraph explaining why you should **never** commit AWS credentials or GitHub tokens to a repository, even a private one.

---

## 14. Key Terms Glossary

- **AWS CLI** – A command-line tool for interacting with AWS services using your IAM credentials.
- `aws configure` – Command that stores your access key, secret key, region, and output format locally for the CLI to use.
- **Personal Access Token (PAT)** – A revocable, scoped credential used instead of a password for Git operations on GitHub.
- **Credential helper** – A Git feature that temporarily caches credentials so you don't retype them on every push/pull.
- **Persistent container** – A stopped (not removed) Docker container that retains its filesystem changes when restarted.

---

*Instructor note: Remind students to revoke/regenerate tokens at the end of the semester.*